1. **Find square root of a number. Can we use Divide & Conquer approach for this problem?**

```
public class SquareRoot {
    public static int floorSqrt(int n) {
        int low = 1, high = n, ans = 0;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (mid <= n / mid) {
                ans = mid;
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return ans;
    }
    public static void main(String[] args) {
        int n = 11;
        System.out.println(floorSqrt(n));
    }
}
```

Output:

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                    Debug: p1

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:62711 -XX:+ShowCodeD
etailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3585d\\redhat.java\\jdt_ws\\pertical_
52d5471d\\bin p1
3
```

2. **Determine smallest divisor of an integer.**

```
public class SmallestDivisor {
    public static int findSmallestDivisor(int n) {
        if (n <= 1) return n; // Edge case
        for (int i = 2; i*i <= n; i++) {
            if (n % i == 0)
                return i;
        }
        return n;
}
    public static void main(String[] args) {
        int n = 15;
        System.out.println(findSmallestDivisor(n));
```

```
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:627
70 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin SmallestDivisor2
3
```

3. For a given value of n, generate prime numbers <= n (more than one algorithms are possible)

```
public class PrimesTrialDivision {
    public static boolean isPrime(int num) {
        if (num < 2) return false;
        for (int i=2; i*i<=num; i++)
            if (num % i == 0) return false;
        return true;
    }

    public static void main(String[] args) {
        int n = 30;
        for (int i = 2; i <= n; i++) {
            if (isPrime(i)) System.out.print(i + " ");
        }
        // Output: 2 3 5 7 11 13 17 19 23 29
    }
}
```

Output:

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:628
14 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin p3
2 3 5 7 11 13 17 19 23 29
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
```

4. Find Xn. Iterative and recursive algorithms are possible.

```
public class FindXn {
    public static int recursiveXn(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        return recursiveXn(n - 1) + recursiveXn(n - 2);
    }
    public static void main(String[] args) {
        int n = 10;
        System.out.println(recursiveXn(n));
    }
}
```

Output:

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                    Debug: FindXn

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:62833 -XX:+ShowCodeD
etailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3585d\\redhat.java\\jdt_ws\\pertical_
52d5471d\\bin FindXn
55
```

5. Find Factorial of n. Iterative and recursive algorithms are possible.

```
public class Factorial {
   public static long recursiveFactorial(int n) {
      if (n == 0 || n == 1)
         return 1;
      return n * recursiveFactorial(n - 1);
   }

   public static void main(String[] args) {
      int n = 5;
      System.out.println(recursiveFactorial(n));
 }
}
```

Output:

```
PROBLEMS 10   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:628
52 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin Factorial
120
```

6. Generate Fibonacci series up to n terms Iterative and recursive algorithms are possible.

```
public class FibonacciIterative {
   public static void fibonacciIterative(int n) {
      int a = 0, b = 1;
      for (int i = 1; i <= n; i++) {
         System.out.print(a + " ");
         int next = a + b;
         a = b;
         b = next;
      }
   }

   public static void main(String[] args) {
      int n = 10;
      fibonacciIterative(n);
   }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:628
71 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin FibonacciIterative
0 1 1 2 3 5 8 13 21 34
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
```

7. Program for finding maximum and minimum number using Divide and conquer.

```
public class MaxMinDivideConquer {
    static class Pair {
      int max;
      int min;
   }

   static Pair findMaxMin(int[] arr, int low, int high) {
      Pair result = new Pair();

          if (low == high) {
         result.max = arr[low];
         result.min = arr[low];
         return result;
      }

          if (high == low + 1) {
         if (arr[low] < arr[high]) {
            result.min = arr[low];
            result.max = arr[high];
         } else {
            result.min = arr[high];
            result.max = arr[low];
         }
         return result;
      }

      int mid = (low + high) / 2;
      Pair left = findMaxMin(arr, low, mid);
      Pair right = findMaxMin(arr, mid + 1, high);
     result.max = Math.max(left.max, right.max);

      result.min = Math.min(left.min, right.min);
      return result;
   }
   public static void main(String[] args) {
      int[] arr = {6, 4, 26, 14, 33, 64, 46};
      Pair result = findMaxMin(arr, 0, arr.length - 1);
```

```
        System.out.println("Maximum element is: " + result.max);
        System.out.println("Minimum element is: " + result.min);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:639
40 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin MaxMinDivideConquer
Maximum element is: 64
Minimum element is: 4
```

8. Implement Recursive Binary search and Linear search and determine the time taken to search an element.

```
public class SearchComparison {

    public static int recursiveBinarySearch(int[] arr, int low, int high, int target) {
        if (low > high)
            return -1;
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] > target)
            return recursiveBinarySearch(arr, low, mid - 1, target);
        else
            return recursiveBinarySearch(arr, mid + 1, high, target);
    }

    public static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target)
                return i;
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] sortedArr = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
            int target = 7;

        long startTime = System.nanoTime();
        int binaryResult = recursiveBinarySearch(sortedArr, 0, sortedArr.length - 1,
target);
        long binaryTime = System.nanoTime() - startTime;
```

```
        long startLinear = System.nanoTime();
        int linearResult = linearSearch(sortedArr, target);
        long linearTime = System.nanoTime() - startLinear;
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:65
21 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e35
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin SearchComparison
Recursive Binary Search: Index = 3, Time taken = 1600 ns
Linear Search: Index = 3, Time taken = 700 ns
```

9. Sort a given sequence of numbers using (a) Bubble Sort, and (b) Merge Sort

(a) Bubble Sort

```
public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        boolean swapped;
        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }
            if (!swapped) break; // Array is sorted
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        for (int num : arr) System.out.print(num + " ");
      }
}
```

Output:

```
C:\Users\daxp1\OneDrive\Desktop\MCA sem3\DAA\pertical\PrimAlgorithm.java
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:65
59 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e35
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin BubbleSort
11 12 22 25 34 64 90
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$
```

(b) Merge Sort

```
public class MergeSort {
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    public static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        System.arraycopy(arr, left, L, 0, n1);
        System.arraycopy(arr, mid + 1, R, 0, n2);

        int i = 0, j = 0, k = left;

        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }

        while (i < n1) arr[k++] = L[i++];
        while (j < n2) arr[k++] = R[j++];
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        mergeSort(arr, 0, arr.length - 1);
        for (int num : arr) System.out.print(num + " ");
          }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:651
78 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin MergeSort
11 12 22 25 34 64 90
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$
```

10. **Generate permutations of given n numbers. Iterative and recursive algorithms are possible.**

1. **Recursive Algorithm (backtracking):**

```
import java.util.*;
public class PermutationsRecursive {

    public static void permute(int[] arr, int start, List<List<Integer>> result) {
        if (start == arr.length) {
            List<Integer> permutation = new ArrayList<>();
            for (int num : arr) permutation.add(num);
            result.add(permutation);
        } else {
            for (int i = start; i < arr.length; i++) {
                swap(arr, start, i);
                permute(arr, start + 1, result);
                swap(arr, start, i);  // backtrack
            }
        }
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        List<List<Integer>> result = new ArrayList<>();
        permute(arr, 0, result);
        for (List<Integer> perm : result)
            System.out.println(perm);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:535
22 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin PermutationsRecursive
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 2, 1]
[3, 1, 2]
```

2. **Iterative Algorithm (using Heap's Algorithm):**

```
import java.util.*;
```

```
public class PermutationsIterative {

    public static void heapPermutation(int[] arr, int n, List<List<Integer>> result) {
        int[] c = new int[n];
        Arrays.fill(c, 0);
        result.add(toList(arr));
        int i = 0;
        while (i < n) {
            if (c[i] < i) {
                if (i % 2 == 0) {
                    swap(arr, 0, i);
                } else {
                    swap(arr, c[i], i);
                }
                result.add(toList(arr));
                c[i]++;
                i = 0;
            } else {
                c[i] = 0;
                i++;
            }
        }
    }

    private static List<Integer> toList(int[] arr) {
        List<Integer> list = new ArrayList<>();
        for (int num : arr) list.add(num);
        return list;
    }

    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        List<List<Integer>> result = new ArrayList<>();
        heapPermutation(arr, arr.length, result);
        for (List<Integer> perm : result)
            System.out.println(perm);
    }
```

```
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  cd c:\\Users\\daxp1\\OneDrive\\Desktop\\MCA\ sem3\\DAA\\pertical ; /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.ex
b:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:53550 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\A
aming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3585d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin p10
[1, 2, 3]
[2, 1, 3]
[3, 1, 2]
[1, 3, 2]
[2, 3, 1]
[3, 2, 1]
```

11.Implement Strassen's algorithm for matrix multiplication

```
public class Strassen {

    private static int[][] add(int[][] A, int[][] B) {
        int n = A.length;
        int[][] C = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                C[i][j] = A[i][j] + B[i][j];
        return C;
    }
    private static int[][] subtract(int[][] A, int[][] B) {
        int n = A.length;
        int[][] C = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                C[i][j] = A[i][j] - B[i][j];
        return C;
    }
    private static void split(int[][] P, int[][] C, int iB, int jB) {
        for (int i = 0, i2 = iB; i < C.length; i++, i2++)
            for (int j = 0, j2 = jB; j < C.length; j++, j2++)
                C[i][j] = P[i2][j2];
    }
    private static void join(int[][] C, int[][] P, int iB, int jB) {
        for (int i = 0, i2 = iB; i < C.length; i++, i2++)
            for (int j = 0, j2 = jB; j < C.length; j++, j2++)
                P[i2][j2] = C[i][j];
    }
    public static int[][] multiply(int[][] A, int[][] B) {
        int n = A.length;
        int[][] R = new int[n][n];

        if (n == 1) {
            R[0] = A * B;
```

```
    } else {
        int newSize = n / 2;
        int[][] A11 = new int[newSize][newSize];
        int[][] A12 = new int[newSize][newSize];
        int[][] A21 = new int[newSize][newSize];
        int[][] A22 = new int[newSize][newSize];
        int[][] B11 = new int[newSize][newSize];
        int[][] B12 = new int[newSize][newSize];
        int[][] B21 = new int[newSize][newSize];
        int[][] B22 = new int[newSize][newSize];

        split(A, A11, 0, 0);
        split(A, A12, 0, newSize);
        split(A, A21, newSize, 0);
        split(A, A22, newSize, newSize);

        split(B, B11, 0, 0);
        split(B, B12, 0, newSize);
        split(B, B21, newSize, 0);
        split(B, B22, newSize, newSize);

        int[][] M1 = multiply(add(A11, A22), add(B11, B22));
        int[][] M2 = multiply(add(A21, A22), B11);
        int[][] M3 = multiply(A11, subtract(B12, B22));
        int[][] M4 = multiply(A22, subtract(B21, B11));
        int[][] M5 = multiply(add(A11, A12), B22);
        int[][] M6 = multiply(subtract(A21, A11), add(B11, B12));
        int[][] M7 = multiply(subtract(A12, A22), add(B21, B22));

        int[][] C11 = add(subtract(add(M1, M4), M5), M7);
        int[][] C12 = add(M3, M5);
        int[][] C21 = add(M2, M4);
        int[][] C22 = add(subtract(add(M1, M3), M2), M6);

        join(C11, R, 0, 0);
        join(C12, R, 0, newSize);
        join(C21, R, newSize, 0);
        join(C22, R, newSize, newSize);
    }
    return R;
}

public static void printMatrix(int[][] matrix) {
    for (int[] row : matrix) {
```

```
            for (int val : row)
                System.out.print(val + " ");
            System.out.println();
        }
    }

    public static void main(String[] args) {
      int[][] A = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12},
            {13, 14, 15, 16}
        };
        int[][] B = {
            {16, 15, 14, 13},
            {12, 11, 10, 9},
            {8, 7, 6, 5},
            {4, 3, 2, 1}
        };

        int[][] result = multiply(A, B);
        System.out.println("Product of matrices A and B:");
        printMatrix(result);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:53565 -XX:+ShowCodeD
etailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3585d\\redhat.java\\jdt_ws\\pertical_
52d5471d\\bin Strassen
Product of matrices A and B:
80 70 60 50
240 214 188 162
400 358 316 274
560 502 444 386
```

12. Implement knapsack using greedy algorithm.

```
import java.util.Arrays;
import java.util.Comparator;

class Item {
    int weight;
    int profit;
    double ratio;

    Item(int profit, int weight) {
        this.profit = profit;
        this.weight = weight;
        ratio = (double) profit / weight;
```

```
    }
}

public class GreedyKnapsack {

    public static double fractionalKnapsack(int capacity, Item[] items) {

        Arrays.sort(items, new Comparator<Item>() {
            public int compare(Item a, Item b) {
                return Double.compare(b.ratio, a.ratio);
            }
        });

        double totalProfit = 0;
        int remainingCapacity = capacity;

        for (Item item : items) {
            if (item.weight <= remainingCapacity) {
                remainingCapacity -= item.weight;
                totalProfit += item.profit;
            } else {
                // Take fraction of item
                totalProfit += item.ratio * remainingCapacity;
                break;
            }
        }
        return totalProfit;
    }

    public static void main(String[] args) {
        Item[] items = {new Item(60, 10), new Item(100, 20), new Item(120, 30)};
        int capacity = 50;

        double maxProfit = fractionalKnapsack(capacity, items);
        System.out.println("Maximum profit in Knapsack = " + maxProfit);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:535
86 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin GreedyKnapsack
Maximum value in Knapsack = 240.0
```

13. Implement Matrix multiplication using Dynamic Programming algorithm.

```
import java.util.Arrays;

public class MatrixChainMultiplication {

    static int matrixChainOrder(int[] p) {
        int n = p.length;
        int[][] dp = new int[n][n];

        for (int i = 1; i < n; i++) {
            dp[i][i] = 0;
        }
        for (int L = 2; L < n; L++) {
            for (int i = 1; i < n - L + 1; i++) {
                int j = i + L - 1;
                dp[i][j] = Integer.MAX_VALUE;
                for (int k = i; k <= j - 1; k++) {
                    int q = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
                    if (q < dp[i][j]) {
                        dp[i][j] = q;
                    }
                }
            }
        }
        return dp[1][n - 1];
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4}; // Dimensions of matrices: A1=1x2, A2=2x3, A3=3x4
        int minMultiplications = matrixChainOrder(arr);
        System.out.println("Minimum number of multiplications is: " +
minMultiplications);
        }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$ /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:536
04 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin MatrixChainMultiplication
Minimum number of multiplications is: 18
```

14. Implement Rod-cutting problem using Dynamic Programming algorithm.

```
import java.util.Arrays;
```

```
public class MatrixChainMultiplication {

  static int matrixChainOrder(int[] p) {
      int n = p.length;
      int[][] dp = new int[n][n];

      for (int i = 1; i < n; i++) {
         dp[i][i] = 0;
      }

      for (int L = 2; L < n; L++) {
         for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            dp[i][j] = Integer.MAX_VALUE;
            for (int k = i; k <= j - 1; k++) {
               int q = dp[i][k] + dp[k + 1][j] + p[i - 1] * p[k] * p[j];
               if (q < dp[i][j]) {
                  dp[i][j] = q;
               }
            }
         }
      }
      return dp[1][n - 1];
   }

   public static void main(String[] args) {
      int[] arr = {1, 2, 3, 4};
      int minMultiplications = matrixChainOrder(arr);
      System.out.println("Minimum number of multiplications is: " +
minMultiplications);
   }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$ /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:536
31 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin p14
Minimum number of multiplications is: 18
```

15. Implement LCS using Dynamic Programming algorithm.

```
public class LongestCommonSubsequence {

   public static int lcs(String X, String Y) {
      int m = X.length();
      int n = Y.length();
```

```
        int[][] dp = new int[m + 1][n + 1];
        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0 || j == 0)
                    dp[i][j] = 0;
                else if (X.charAt(i - 1) == Y.charAt(j - 1))
                    dp[i][j] = dp[i - 1][j - 1] + 1;
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
    public static void main(String[] args) {
        String X = "AGGTAB";
        String Y = "GXTXAYB";

        int length = lcs(X, Y);
        System.out.println("Length of LCS is " + length);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:5
47 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin LongestCommonSubsequence
Length of LCS is 4
```

16. Breadth First Search (BFS) in a binary tree.

```
import java.util.LinkedList;
import java.util.Queue;

class TreeNode {
    int val;
    TreeNode left, right;

    TreeNode(int item) {
        val = item;
        left = right = null;
    }
}

public class BinaryTreeBFS {
    TreeNode root;
   public void bfs() {
        if (root == null)
```

```
            return;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            System.out.print(node.val + " ");

            if (node.left != null)
                queue.add(node.left);
            if (node.right != null)
                queue.add(node.right);
        }
    }

    public static void main(String[] args) {
        BinaryTreeBFS tree = new BinaryTreeBFS();
        tree.root = new TreeNode(10);
        tree.root.left = new TreeNode(20);
        tree.root.right = new TreeNode(30);
        tree.root.left.left = new TreeNode(40);
        tree.root.left.right = new TreeNode(50);
        tree.root.right.right = new TreeNode(60);

        System.out.print("BFS Traversal: ");
        tree.bfs();
          }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:536
63 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin BinaryTreeBFS
BFS Traversal: 10 20 30 40 50 60
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
```

17. Depth First Search (DFS) in a binary tree.

```
class TreeNode {
    int val;
    TreeNode left, right;

    TreeNode(int val) {
        this.val = val;
        left = right = null;
    }
}
```

```
public class BinaryTreeDFS {

    public void dfsPreorder(TreeNode node) {
        if (node == null) return;
        System.out.print(node.val + " ");
        dfsPreorder(node.left);
        dfsPreorder(node.right);
    }

    public void dfsInorder(TreeNode node) {
        if (node == null) return;
        dfsInorder(node.left);
        System.out.print(node.val + " ");
        dfsInorder(node.right);
    }

    public void dfsPostorder(TreeNode node) {
        if (node == null) return;
        dfsPostorder(node.left);
        dfsPostorder(node.right);
        System.out.print(node.val + " ");
    }

    public static void main(String[] args) {
        BinaryTreeDFS tree = new BinaryTreeDFS();

        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);

        System.out.print("Preorder DFS: ");
        tree.dfsPreorder(root); // Output: 1 2 4 5 3
        System.out.println();

        System.out.print("Inorder DFS: ");
        tree.dfsInorder(root);  // Output: 4 2 5 1 3
        System.out.println();

        System.out.print("Postorder DFS: ");
        tree.dfsPostorder(root);
        System.out.println();
    }
```

```
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:58564 -XX:+ShowCodeD
etailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e3585d\\redhat.java\\jdt_ws\\pertical_
52d5471d\\bin BinaryTreeDFS
Preorder DFS: 1 2 4 5 3
Inorder DFS: 4 2 5 1 3
Postorder DFS: 4 5 2 3 1
```

18. Prim's algorithm to find minimum cost tree (shortest path in a tree).

```
import java.util.Arrays;

public class PrimAlgorithm {

    private static final int V = 5;

    private int minKey(int key[], boolean mstSet[]) {
        int min = Integer.MAX_VALUE, minIndex = -1;

        for (int v = 0; v < V; v++) {
            if (!mstSet[v] && key[v] < min) {
                min = key[v];
                minIndex = v;
            }
        }
        return minIndex;
    }
    public void primMST(int graph[][]) {
        int parent[] = new int[V];
       int key[] = new int[V];
        boolean mstSet[] = new boolean[V];
        Arrays.fill(key, Integer.MAX_VALUE);

        key[0] = 0;
        parent = -1;
        for (int count = 0; count < V - 1; count++) {
            int u = minKey(key, mstSet);
            mstSet[u] = true;

            for (int v = 0; v < V; v++) {
                if (graph[u][v] != 0 && !mstSet[v] && graph[u][v] < key[v]) {
                    parent[v] = u;
                    key[v] = graph[u][v];
                }
            }
        }
```

```
        System.out.println("Edge \tWeight");
        for (int i = 1; i < V; i++) {
            System.out.println(parent[i] + " - " + i + "\t" + graph[i][parent[i]]);
        }
    }

    public static void main(String[] args) {
        PrimAlgorithm t = new PrimAlgorithm();
        int graph[][] = new int[][] {
            {0, 9, 0, 0, 0},
            {9, 0, 75, 19, 42},
            {0, 75, 0, 51, 66},
            {0, 19, 51, 0, 31},
            {0, 42, 66, 31, 0}
        };

        t.primMST(graph);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:585
86 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin PrimAlgorithm
Edge    Weight
0 - 1   9
3 - 2   51
1 - 3   19
3 - 4   31
```

19. Kruskal's algorithm to find minimum cost tree (shortest path in a tree).

```
import java.util.Arrays;

public class KruskalAlgorithm {
    class Edge implements Comparable<Edge> {
        int src, dest, weight;
        public int compareTo(Edge compareEdge) {
            return this.weight - compareEdge.weight;
        }
    }
    class Subset {
        int parent, rank;
    }

    int vertices, edges;
    Edge[] edge;
    KruskalAlgorithm(int v, int e) {
        vertices = v;
```

```
    edges = e;
    edge = new Edge[edges];
    for (int i = 0; i < e; i++) {
        edge[i] = new Edge();
    }
}

int find(Subset[] subsets, int i) {
    if (subsets[i].parent != i) {
        subsets[i].parent = find(subsets, subsets[i].parent);
    }
    return subsets[i].parent;
}

void union(Subset[] subsets, int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank) {
        subsets[xroot].parent = yroot;
    } else if (subsets[xroot].rank > subsets[yroot].rank) {
        subsets[yroot].parent = xroot;
    } else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

void kruskalMST() {
    Edge[] result = new Edge[vertices];  // This will store the resultant MST
    int e = 0;  // Index for result[]
    int i = 0;  // Index for sorted edges

    for (i = 0; i < vertices; i++) {
        result[i] = new Edge();
    }


    Arrays.sort(edge);

    Subset[] subsets = new Subset[vertices];
    for (i = 0; i < vertices; i++) {
        subsets[i] = new Subset();
        subsets[i].parent = i;
```

```
            subsets[i].rank = 0;
        }

        i = 0;
        while (e < vertices - 1 && i < edges) {
            Edge nextEdge = edge[i++];

            int x = find(subsets, nextEdge.src);
            int y = find(subsets, nextEdge.dest);
            if (x != y) {
                result[e++] = nextEdge;
                union(subsets, x, y);
            }
        }

             System.out.println("Edge \tWeight");
        for (i = 0; i < e; i++) {
            System.out.println(result[i].src + " - " + result[i].dest + "\t" + result[i].weight);
        }
    }

    public static void main(String[] args) {
        int vertices = 6;  // Number of vertices
        int edges = 8;     // Number of edges
        KruskalAlgorithm graph = new KruskalAlgorithm(vertices, edges);

        // Adding edges: src, dest, weight
        graph.edge[0].src = 0; graph.edge.dest = 1; graph.edge.weight = 4;
        graph.edge[11].src = 0; graph.edge[11].dest = 2; graph.edge[11].weight = 4;
        graph.edge[12].src = 1; graph.edge[12].dest = 2; graph.edge[12].weight = 2;
        graph.edge[13].src = 2; graph.edge[13].dest = 3; graph.edge[13].weight = 3;
        graph.edge[14].src = 2; graph.edge[14].dest = 5; graph.edge[14].weight = 2;
        graph.edge[15].src = 2; graph.edge[15].dest = 4; graph.edge[15].weight = 4;
        graph.edge[16].src = 3; graph.edge[16].dest = 4; graph.edge[16].weight = 3;
        graph.edge[17].src = 5; graph.edge[17].dest = 4; graph.edge[17].weight = 3;

        graph.kruskalMST();
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:586
00 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin KruskalAlgorithm
Edge    Weight
1 - 2   2
2 - 5   2
2 - 3   3
3 - 4   3
0 - 1   4
```

20. Implement Bellman-Ford Single Source Shortest Path Algorithm.

```
import java.util.Arrays;

public class BellmanFord {

    static class Edge {
     int source, destination, weight;

     Edge(int s, int d, int w) {
        source = s;
        destination = d;
        weight = w;
     }
  }

  int V, E;
  Edge[] edges;
  public BellmanFord(int vertices, int edgesCount) {
     V = vertices;
     E = edgesCount;
     edges = new Edge[E];
  }

  public void bellmanFordAlgo(int src) {
     int[] dist = new int[V];
     Arrays.fill(dist, Integer.MAX_VALUE);
     dist[src] = 0;

     for (int i = 1; i < V; i++) {
        for (int j = 0; j < E; j++) {
           int u = edges[j].source;
           int v = edges[j].destination;
           int weight = edges[j].weight;
           if (dist[u] != Integer.MAX_VALUE && dist[u] + weight < dist[v]) {
              dist[v] = dist[u] + weight;
           }
        }
     }
```

```
        for (int j = 0; j < E; j++) {
            int u = edges[j].source;
            int v = edges[j].destination;
            int weight = edges[j].weight;
            if (dist[u] != Integer.MAX_VALUE && dist[u] + weight < dist[v]) {
                System.out.println("Graph contains negative weight cycle");
                return;
            }
        }
        printDistances(dist);
    }
    private void printDistances(int[] dist) {
        System.out.println("Vertex Distance from Source:");
        for (int i = 0; i < dist.length; i++) {
            System.out.println(i + "\t\t" + dist[i]);
        }
    }
    public static void main(String[] args) {
        int V = 5;
        int E = 8;
BellmanFord graph = new BellmanFord(V, E);
         graph.edges[0] = new Edge(0, 1, -1);
        graph.edges[10] = new Edge(0, 2, 4);
        graph.edges[11] = new Edge(1, 2, 3);
        graph.edges[12] = new Edge(1, 3, 2);
        graph.edges[13] = new Edge(1, 4, 2);
        graph.edges[14] = new Edge(3, 2, 5);
        graph.edges[15] = new Edge(3, 1, 1);
        graph.edges[16] = new Edge(4, 3, -3);
        graph.bellmanFordAlgo(0);
    }
}
```

Output:

```
daxp1@Daxpanchal MINGW64 ~/OneDrive/Desktop/MCA sem3/DAA/pertical (master)
$  /usr/bin/env C:\\Program\ Files\\Java\\jdk-23.0.1\\bin\\java.exe -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:586
42 -XX:+ShowCodeDetailsInExceptionMessages -cp C:\\Users\\daxp1\\AppData\\Roaming\\Code\\User\\workspaceStorage\\fde94ad79d55421ecd1bb79c79e358
5d\\redhat.java\\jdt_ws\\pertical_52d5471d\\bin BellmanFord
Vertex Distance from Source:
0               0
1               -1
2               2
3               -2
4               1
```